

# Virtualização de Sistemas

Infraestruturas de Computação na Nuvem, Aula 03

---

**Catarina Silva**

ESTGA, Universidade de Aveiro | 2026

## Agenda (1/3)

---

- Conceito de virtualização e níveis de aplicação.
- Requisitos de Popek e Goldberg e classificação de instruções.
- Anéis de privilégio e a limitação da arquitetura x86.
- Estratégias de virtualização de CPU e arquiteturas de hypervisor.

## Agenda (2/3)

---

- Virtualização de memória: tabelas sombra e paginação em dois níveis.
- Sobrecompromisso de memória: KSM e ballooning.
- Virtualização de I/O: emulação, virtio, passthrough e SR-IOV.
- Armazenamento virtual: formatos, provisionamento fino e instantâneos.

## Agenda (3/3)

---

- Migração em execução e sobrecarga de virtualização.
- KVM, LXC e a plataforma Proxmox VE.
- Virtualização completa e virtualização ao nível do sistema operativo.
- Enquadramento do laboratório de contentores.

# Conceito de Virtualização

---

- Designa-se por **virtualização** a construção de uma representação abstrata de um recurso computacional, funcionalmente equivalente ao recurso físico mas dissociada da sua implementação.
- A camada que realiza esta abstração assume três funções: *traduzir* referências virtuais em físicas, *arbitrar* o acesso concorrente e *isolar* as instâncias entre si.
- Propriedades decorrentes: consolidação de cargas sobre menos equipamento, isolamento de falhas entre instâncias e portabilidade face ao hardware subjacente.
- A virtualização constitui o mecanismo que torna operacional o modelo de partilha de recursos estabelecido na Aula 01.

# Níveis de Virtualização

---

- **Nível de hardware:** o hypervisor apresenta um conjunto de instruções e dispositivos virtuais. Cada instância executa núcleo próprio e sistema operativo completo. Exemplos: KVM, Xen, ESXi.
- **Nível de sistema operativo:** o núcleo é único e partilhado, apresentando a cada instância uma vista isolada dos seus recursos. Exemplos: LXC, Docker, Solaris Zones.
- **Nível de aplicação:** a abstracção situa-se sobre o sistema operativo, executando código destinado a uma máquina abstrata. Exemplos: JVM, CLR.
- O critério distintivo é a *fronteira de duplicação*: o que é replicado por instância e o que permanece partilhado. Determina a sobrecarga e o grau de isolamento.

# Os Requisitos de Popek e Goldberg

---

- Formulados em 1974, estabelecem três propriedades exigíveis a um monitor de máquina virtual.
- **Equivalência:** um programa executado sob o monitor apresenta comportamento indistinguível do que apresentaria em execução direta sobre o hardware.
- **Controle de recursos:** o monitor mantém domínio completo sobre os recursos físicos, não podendo uma instância aceder a recursos não atribuídos.
- **Eficiência:** uma fração estatisticamente dominante das instruções executa diretamente no processador, sem intervenção do monitor.
- O terceiro requisito distingue virtualização de emulação: um emulador satisfaz os dois primeiros, mas interpreta a totalidade das instruções.

# Classificação de Instruções

---

- **Privilegiadas:** geram exceção quando executadas em modo utilizador, sendo admitidas apenas em modo supervisor.
- **Sensíveis:** subdividem-se em sensíveis ao *controlo*, que alteram a configuração de recursos do sistema, e sensíveis ao *comportamento*, cujo resultado depende dessa configuração.
- **Inócuas:** as restantes, sem efeito sobre o estado global, executáveis sem mediação.
- **Teorema de Popek e Goldberg:** um monitor de máquina virtual pode ser construído se o conjunto das instruções sensíveis estiver contido no das privilegiadas. A condição é verificável sobre a especificação da arquitetura, independentemente de qualquer implementação.

# Anéis de Privilégio e a Limitação do x86

---

- A arquitetura x86 define quatro anéis de privilégio, do anel 0, reservado ao núcleo, ao anel 3, destinado a aplicações.
- Sob virtualização, o anel 0 é ocupado pelo monitor, sendo o núcleo convidado deslocado para um anel de menor privilégio, técnica designada por *ring deprivileging*.
- A arquitetura x86 anterior a 2005 não satisfazia a condição do teorema: dezassete instruções eram sensíveis sem serem privilegiadas.
- Exemplo característico: POPF, que ao restaurar o registo de estado em anel 3 ignora silenciosamente a alteração do bit de interrupções. A falha é adversa por não ser detetável: a instrução não falha, produz resultado incorreto.

## Trap-and-Emulate

---

- Mecanismo base da virtualização em arquiteturas que satisfazem a condição do teorema.
- O código convidado executa diretamente no processador. Ao emitir uma instrução privilegiada em modo não privilegiado, o processador gera exceção e transfere o controlo para o monitor.
- O monitor determina a operação pretendida, emula o seu efeito sobre o estado virtual da instância e devolve o controlo ao ponto de interrupção.
- Aplica-se a múltiplos subsistemas: registos de controlo do processador, unidade de gestão de memória, controlador de interrupções, temporizadores e dispositivos periféricos.

# Pergunta 1

---

## Aplicabilidade do teorema

Numa arquitetura hipotética existem instruções que consultam a configuração de recursos do sistema e devolvem resultado distinto consoante o modo de execução, sem gerar exceção em modo utilizador. Que consequência decorre para a construção de um monitor por trap-and-emulate?

- A) Nenhuma. O monitor interceta igualmente essas instruções.
- B) O mecanismo é insuficiente: tais instruções são sensíveis sem serem privilegiadas, pelo que não geram exceção e produzem resultado incorreto sem sinalização.
- C) O monitor deve ser executado em anel 3 para as intercetar.
- D) A arquitetura não admite qualquer forma de virtualização.

## Resposta 1

---

### Opção correta: B

A situação descrita corresponde à violação da condição do teorema: existem instruções sensíveis fora do conjunto das privilegiadas. Não gerando exceção, não são intercetáveis por trap-and-emulate, e a instância obtém informação relativa ao estado real da máquina em vez do seu estado virtual, comprometendo o requisito de equivalência. A opção D é incorreta: a virtualização permanece possível por outras vias, designadamente tradução binária ou paravirtualização, apresentadas em seguida.

# Estratégias de Virtualização de CPU

---

- **Virtualização completa por tradução binária:** o monitor analisa e reescreve o fluxo de instruções do convidado antes da execução, sem exigir alteração do sistema operativo.
- **Paravirtualização:** o núcleo convidado é modificado para invocar explicitamente o monitor nas operações sensíveis, dispensando a intercepção.
- **Virtualização assistida por hardware:** extensões do processador introduzem um modo de execução dedicado, restaurando a validade do trap-and-emulate.
- As três respondem à mesma limitação arquitetural, com compromissos distintos entre desempenho, compatibilidade e necessidade de alterar o convidado.

## Tradução Binária Dinâmica

---

- O monitor examina blocos de código do convidado antes da execução e substitui as instruções problemáticas por sequências equivalentes que transferem o controle.
- A tradução incide apenas sobre código executado em anel 0. O código de aplicação, em anel 3, executa sem modificação.
- Os blocos traduzidos são retidos em memória cache, pelo que o custo é amortizado: código executado repetidamente é traduzido uma única vez.
- Vantagem determinante: admite sistemas operativos convidados não modificados, incluindo sistemas de código fechado. Foi a abordagem que viabilizou comercialmente a virtualização de x86.

# Paravirtualização

---

- O núcleo convidado é modificado de modo a não emitir instruções sensíveis, substituindo-as por invocações explícitas ao monitor, designadas **hypercalls**.
- A interface entre convidado e monitor passa a ser declarada, à semelhança de uma chamada de sistema entre aplicação e núcleo.
- Elimina-se o custo de interceção e de emulação, e o monitor deixa de necessitar de emular hardware que não existe.
- Limitação: exige acesso ao código do sistema operativo convidado e adaptação a cada monitor, o que restringe a sua aplicação a sistemas de código aberto.
- Persiste hoje sobretudo nos controladores paravirtualizados de I/O, matéria retomada adiante.

# Virtualização Assistida por Hardware

---

- As extensões Intel VT-x e AMD-V introduzem dois modos de operação ortogonais aos anéis: **VMX root**, onde executa o monitor, e **VMX non-root**, onde executa o convidado.
- O núcleo convidado executa em anel 0 do modo non-root, pelo que não é necessário deslocá-lo de anel nem reescrever o seu código.
- A transição entre modos é mediada por uma estrutura de controlo em memória, a **VMCS**, que retém o estado de ambos os contextos e determina que eventos provocam saída para o monitor.
- Designa-se por **VM exit** a transição para o monitor e por **VM entry** o regresso. Cada transição tem custo mensurável, pelo que a otimização consiste em reduzir a sua frequência.

## Pergunta 2

---

### Seleção de estratégia

Pretende-se virtualizar um sistema operativo de código fechado, sem possibilidade de modificação, sobre processadores anteriores à introdução das extensões de virtualização. Que estratégia é aplicável?

- A) Paravirtualização, por apresentar melhor desempenho.
- B) Trap-and-emulate puro, por ser o mecanismo de referência.
- C) Tradução binária dinâmica, única das três que dispensa simultaneamente a modificação do convidado e o suporte do processador.
- D) Nenhuma. O cenário descrito não admite virtualização.

## Resposta 2

---

### Opção correta: C

A paravirtualização exige modificação do núcleo convidado, inviável em código fechado. O trap-and-emulate puro pressupõe a validade da condição do teorema, que a arquitetura em causa não satisfaz. As extensões de hardware não se encontram disponíveis. Resta a tradução binária dinâmica, que opera sobre o fluxo de instruções em tempo de execução sem requerer cooperação do convidado nem suporte do processador. Foi precisamente esta a razão da sua adoção comercial na década de 2000.

# Arquiteturas de Hypervisor

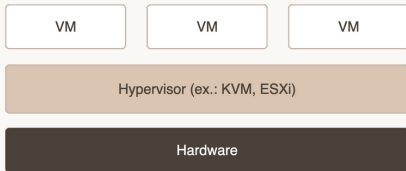
---

- **Tipo 1**, nativo: executa diretamente sobre o hardware, assumindo as funções de escalonamento e gestão de recursos. Exemplos: VMware ESXi, Xen, KVM.
- **Tipo 2**, alojado: executa como processo sobre um sistema operativo anfitrião, que mantém a mediação do acesso ao hardware. Exemplos: VirtualBox, VMware Workstation.
- A distinção reduz-se ao número de camadas interpostas entre a instância e o hardware, com efeito direto sobre a sobrecarga e sobre o determinismo do escalonamento.
- O KVM constitui caso particular: sendo módulo do núcleo Linux, converte o próprio núcleo em hypervisor de tipo 1, mantendo embora as funcionalidades de sistema operativo convencional.

# As Duas Arquiteturas de Hypervisor

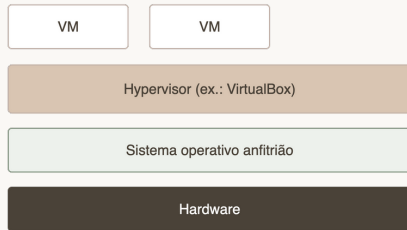
---

**Tipo 1 (bare-metal)**



menos camadas, menor overhead  
— usado em produção

**Tipo 2 (hosted)**



corre como aplicação sobre um SO  
— usado em desenvolvimento

# Virtualização de Memória: o Problema

---

- Um sistema operativo convidado assume dispor de memória física contígua com início no endereço zero, e mantém tabelas de páginas que traduzem endereços virtuais nesse espaço.
- Sob virtualização, esse espaço é ele próprio virtual. Passam a existir três espaços de endereçamento: virtual do convidado, físico do convidado e físico do anfitrião.
- A unidade de gestão de memória do processador realiza uma única tradução, pelo que a composição das duas restantes tem de ser assegurada pelo monitor ou pelo hardware.
- Constitui a fonte de sobrecarga mais significativa da virtualização anterior ao suporte dedicado.

## Tabelas Sombra e Paginação em Dois Níveis

---

- **Tabelas sombra:** o monitor mantém tabelas próprias que traduzem diretamente o endereço virtual do convidado no endereço físico do anfitrião, compondo antecipadamente as duas traduções.
- O custo reside na manutenção: qualquer alteração às tabelas do convidado tem de ser intercetada e propagada, o que exige marcar essas páginas como apenas de leitura e tratar as exceções resultantes.
- **EPT** (Intel) e **NPT** (AMD) transferem a composição para o hardware, que percorre duas hierarquias de tabelas sucessivamente, sem intervenção do monitor.
- A travessia passa a ser bidimensional e, em caso de falha de TLB, mais dispendiosa. Em contrapartida, elimina-se a interceção de escritas, largamente dominante na prática.

## Sobrecompromisso de Memória: KSM e Ballooning

---

- A soma da memória atribuída às instâncias excede frequentemente a memória física instalada, prática admissível por raramente ser utilizada na totalidade em simultâneo.
- **KSM** (*Kernel Samepage Merging*): o anfitrião identifica periodicamente páginas de conteúdo idêntico entre instâncias e substitui-as por uma única página física partilhada, com cópia na escrita.
- **Ballooning**: um controlador instalado no convidado solicita memória ao seu próprio sistema operativo, que a liberta de cache, devolvendo-a ao anfitrião sem que o convidado a considere perdida.
- Ambos exigem cooperação e apresentam custo: o KSM consome processador na comparação de páginas; o ballooning depende da presença do controlador no convidado.

## Pergunta 3

---

### Efeito do sobrecompromisso

Um anfitrião dispõe de 64 GB de memória física e aloja vinte instâncias com 4 GB atribuídos cada, totalizando 80 GB. As instâncias executam a mesma imagem de sistema operativo. Que mecanismo explica a viabilidade desta configuração?

- A) O hypervisor comprime a memória de todas as instâncias.
- B) As instâncias utilizam habitualmente menos do que o atribuído, e o KSM consolida as páginas idênticas resultantes da imagem comum.
- C) A configuração é inviável e o anfitrião recusará iniciar a vigésima instância.
- D) O excesso é integralmente transferido para disco, sem consequências.

## Resposta 3

---

### Opção correta: B

A atribuição constitui um limite superior e não uma reserva efetiva, pelo que o consumo real é tipicamente inferior. Executando as instâncias a mesma imagem, uma fração substancial das suas páginas, designadamente código de núcleo e bibliotecas partilhadas, apresenta conteúdo idêntico, e o KSM consolida-a numa única página física. A opção D descreve a alternativa de recurso, a transferência para disco, que o anfitrião aplica em situação de esgotamento com degradação acentuada do desempenho, e não como regime normal de operação.

## Virtualização de I/O: Emulação e virtio

---

- O I/O constitui o domínio de maior sobrecarga: ao contrário do cálculo, envolve acesso a dispositivos e, por conseguinte, transições frequentes para o monitor.
- **Emulação:** o monitor reproduz o comportamento de um dispositivo real conhecido, por exemplo uma placa de rede Intel e1000. Cada acesso a registo do dispositivo provoca uma transição.
- Vantagem: o convidado utiliza controladores próprios, sem qualquer adaptação. Desvantagem: o número de transições torna o desempenho substancialmente inferior ao nativo.
- **virtio:** interface de dispositivo paravirtualizada, na qual o convidado sabe estar virtualizado e comunica por filas partilhadas em memória, agrupando operações e reduzindo as transições em uma a duas ordens de grandeza.

# Virtualização de I/O: Passthrough e SR-IOV

---

- **Passthrough**: um dispositivo físico é atribuído em exclusivo a uma instância, que o opera com o controlador nativo e desempenho equivalente ao nativo.
- Requer unidade de gestão de memória de I/O (IOMMU) para confinar os acessos diretos à memória da instância. Implica a perda do dispositivo pelo anfitrião e impede a migração em execução.
- **SR-IOV**: extensão do PCI Express pela qual um dispositivo se apresenta como uma função física, gerida pelo anfitrião, e múltiplas funções virtuais, atribuíveis individualmente a instâncias distintas.
- Combina o desempenho do passthrough com a partilha do dispositivo, ao custo de suporte específico no equipamento. Aplicação típica em placas de rede de alto débito e aceleradores.

# Virtualização de Rede

---

- Cada instância dispõe de uma ou mais interfaces virtuais, ligadas a um **comutador virtual** implementado em software no anfitrião.
- O comutador virtual reproduz as funções de um comutador físico, designadamente aprendizagem de endereços, segmentação por VLAN e aplicação de políticas de tráfego.
- O tráfego entre instâncias do mesmo anfitrião não alcança a rede física, o que reduz a latência mas subtrai esse tráfego à observação dos instrumentos de monitorização de rede convencionais.
- A programação centralizada destes comutadores constitui o objeto da Aula 06.

# Formatos de Disco Virtual

---

- **raw**: representação direta do conteúdo do disco, sem metadados. Desempenho máximo e ausência de funcionalidades acessórias.
- **qcow2**: formato nativo do QEMU, com aprovisionamento fino, instantâneos internos, compressão e cifra. A indireção introduzida tem custo de desempenho mensurável.
- **VMDK** e **VHDX**: formatos equivalentes nos ecossistemas VMware e Microsoft.
- **Aprovisionamento fino**: o ficheiro ocupa em armazenamento apenas os blocos efetivamente escritos, e não a capacidade declarada ao convidado.

## Aprovisionamento Fino: Risco Operacional

---

- O provisionamento fino admite declarar capacidade agregada superior à fisicamente instalada, prática análoga ao sobrecompromisso de memória.
- A diferença é determinante: a memória é reclamável, o armazenamento escrito não. Uma instância que preencha o seu disco não liberta espaço ao terminar.
- Consequência do esgotamento do volume subjacente: as escritas falham em todas as instâncias que dele dependem, e não apenas na responsável pelo consumo.
- Requisito operacional: monitorização do espaço real disponível com limiar de alerta, e não do espaço declarado às instâncias.

# Instantâneos, Clones e Modelos

---

- **Instantâneo:** captura do estado da instância num momento determinado, incluindo disco e, opcionalmente, memória e estado do processador.
- Implementado por cópia na escrita: o ficheiro original torna-se apenas de leitura e as escritas subsequentes são registadas num ficheiro diferencial.
- Daqui decorre uma propriedade frequentemente ignorada: instantâneos retidos por períodos longos acumulam cadeias de ficheiros diferenciais que degradam o desempenho de leitura e complicam a recuperação.
- **Clone ligado** partilha as camadas de base com a origem; **clone completo** constitui cópia independente. Um **modelo** é uma instância convertida em imagem apenas de leitura para instanciação repetida.

## Pergunta 4

---

### Instantâneos como cópia de segurança

Uma equipa mantém instantâneos diários das suas instâncias, retidos durante três meses, e considera-os cópia de segurança. Que objeção se impõe?

- A) Nenhuma. Um instantâneo constitui cópia de segurança completa.
- B) Os instantâneos dependem do ficheiro de base original: a sua perda ou corrupção invalida toda a cadeia, que além disso degrada o desempenho ao acumular.
- C) Os instantâneos só podem ser criados com a instância desligada.
- D) A objeção é apenas de custo de armazenamento.

## Resposta 4

---

### Opção correta: B

Um instantâneo não constitui cópia autónoma: regista diferenças relativamente a um ficheiro de base que permanece indispensável. A perda desse ficheiro, ou do volume que o aloja, invalida a totalidade da cadeia, pelo que não protege contra o cenário que uma cópia de segurança deve cobrir. Acresce a degradação progressiva do desempenho de leitura com o comprimento da cadeia. Uma cópia de segurança exige transferência do conteúdo para suporte independente, com verificação periódica do procedimento de reposição.

## Migração em Execução (1/2)

---

- Consiste na transferência de uma instância em execução entre anfitriões físicos sem interrupção perceptível do serviço.
- O estado a transferir decompõe-se em três elementos: os *ficheiros* de disco, com frequência já acessíveis a ambos os anfitriões por armazenamento partilhado; a *configuração* da instância; e o *contexto* de execução, constituído por memória e estado do processador.
- Fases: reserva de recursos no destino, transferência iterativa da memória com a instância ainda ativa, suspensão breve para transferir o estado residual, e retoma no destino com reconfiguração do encaminhamento de rede.
- Constitui o mecanismo que viabiliza manutenção de hardware sem interrupção de serviço, requisito do Tier III analisado na Aula 02.

## Migração em Execução (2/2)

---

- **Pré-cópia:** a memória é transferida iterativamente enquanto a instância executa; cada iteração transfere as páginas modificadas durante a anterior. A suspensão ocorre quando o residual é suficientemente reduzido.
- A convergência depende da relação entre a taxa de modificação de páginas e a largura de banda disponível. Cargas com escrita intensiva podem não convergir, forçando suspensão com residual elevado.
- **Pós-cópia:** a instância é retomada no destino com estado mínimo, sendo as páginas em falta obtidas por exceção. Reduz o tempo de suspensão, mas uma falha de rede durante a transferência torna a instância irrecuperável.
- A pré-cópia constitui o regime adotado por omissão, precisamente por manter a origem íntegra até à conclusão.

## Pergunta 5

---

### Convergência da migração

Uma instância que executa uma base de dados com escrita intensiva não conclui a migração por pré-cópia, permanecendo em iterações sucessivas sem redução do residual. Qual a causa?

- A) Insuficiência de memória no anfitrião de destino.
- B) A taxa de modificação de páginas aproxima-se ou excede a largura de banda de transferência, pelo que cada iteração encontra tantas páginas modificadas quantas transferiu.
- C) O formato de disco é incompatível com migração.
- D) A pré-cópia não admite instâncias com base de dados.

## Resposta 5

---

### Opção correta: B

A pré-cópia converge apenas quando a largura de banda excede a taxa de modificação de páginas. Não se verificando essa condição, cada iteração encontra um conjunto de páginas modificadas de dimensão comparável ao que acabou de transferir, e o residual não diminui. As respostas práticas consistem em aumentar a largura de banda dedicada, impor um limite de iterações após o qual se força a suspensão, aceitando indisponibilidade mais prolongada, ou recorrer à pós-cópia.

# Sobrecarga de Virtualização

---

- A camada de abstração introduz custo mensurável, hoje reduzido mas não nulo, concentrado nas transições para o monitor e na tradução de endereços.
- Cargas dominadas por cálculo aproximam-se do desempenho nativo, por executarem maioritariamente instruções inócuas sem mediação.
- Cargas dominadas por I/O ou sensíveis à latência apresentam degradação superior, decorrente da frequência das transições.
- Cenários em que a execução direta sobre o hardware permanece justificada: bases de dados de elevado débito, computação de alto desempenho e sistemas com requisitos de latência determinística.

# KVM

---

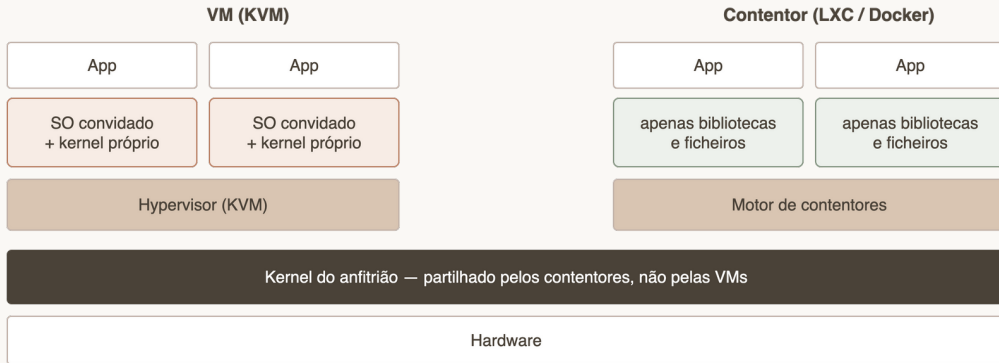
- O **KVM** é um módulo do núcleo Linux que expõe as extensões de virtualização do processador, convertendo o núcleo em hypervisor de tipo 1.
- A arquitetura reparte responsabilidades: o KVM trata da execução do convidado e da gestão de memória; o **QEMU**, em espaço de utilizador, trata da emulação de dispositivos.
- Cada instância corresponde, no anfitrião, a um processo convencional, escalonado pelo escalonador do Linux e observável pelos instrumentos habituais do sistema.
- Constitui a base tecnológica do Proxmox VE e de grande parte das infraestruturas de cloud assentes em Linux.

# LXC e a Virtualização ao Nível do Sistema Operativo

---

- O **LXC** constitui instâncias que partilham o núcleo do anfitrião, dispondo de sistema de ficheiros, espaço de processos e pilha de rede próprios.
- Não existe hardware virtual nem núcleo convidado, do que decorrem tempo de arranque na ordem dos segundos e consumo de memória limitado ao espaço de utilizador.
- O isolamento assenta em mecanismos do núcleo Linux, designadamente *namespaces* e *cgroups*, objeto da Aula 04.
- Restrições correspondentes: apenas sistemas Linux são executáveis, e o isolamento é inferior ao de uma máquina virtual por partilha da superfície de ataque do núcleo.

# O que Cada Modelo Duplica



É por isso que o contentor mostra o mesmo kernel do anfitrião e a VM não.

## Comparação dos Dois Modelos

---

|                   | <b>Máquina virtual</b> | <b>Contentor</b>           |
|-------------------|------------------------|----------------------------|
| Núcleo            | Próprio por instância  | Partilhado com o anfitrião |
| Arranque          | Dezenas de segundos    | Segundos ou menos          |
| Sobrecarga        | Núcleo e dispositivos  | Praticamente nula          |
| Sistema convidado | Qualquer               | Apenas Linux               |
| Isolamento        | Fronteira de hardware  | Fronteira de núcleo        |
| Densidade típica  | Dezenas por anfitrião  | Centenas por anfitrião     |

- Os modelos não são alternativos mas complementares: é prática corrente executar contentores dentro de máquinas virtuais, combinando o isolamento das primeiras com a densidade dos segundos.

## Pergunta 6

---

### Isolamento entre clientes distintos

Um fornecedor pretende alojar cargas de clientes distintos, sem relação de confiança entre si, no mesmo equipamento físico. Que modelo se justifica como fronteira primária de isolamento?

- A) Contentores, por apresentarem menor sobrecarga e maior densidade.
- B) Máquinas virtuais, por a fronteira de isolamento ser assegurada pelo hardware e não pelo núcleo partilhado, cuja superfície de ataque é comum a todos os contentores.
- C) Indiferente: ambos oferecem isolamento equivalente.
- D) Contentores, desde que executados com utilizador não privilegiado.

## Resposta 6

---

### Opção correta: B

Contentores no mesmo anfitrião partilham o núcleo, pelo que uma vulnerabilidade que permita escapar ao isolamento compromete todas as instâncias. A máquina virtual estabelece a fronteira ao nível do hardware, com superfície de ataque substancialmente reduzida. A opção D descreve uma medida de mitigação pertinente, mas que não altera a natureza da fronteira. A prática corrente na indústria consiste em combinar ambos: uma máquina virtual por cliente, com contentores no seu interior.

# Proxmox VE

---

- Plataforma de virtualização que integra numa interface única os dois modelos: KVM para máquinas virtuais e LXC para contentores de sistema.
- Gestão por interface web ou por linha de comandos, através de `qm` para instâncias KVM e `pct` para contentores LXC.
- Suporta instantâneos, clones, modelos e migração entre nós, concretizando os mecanismos analisados nesta aula.
- Constitui a plataforma que sustenta o trabalho de projeto da segunda metade do semestre.

# Laboratório: Primeiro Contacto com Contentores

---

## Trabalho em equipamento próprio

O laboratório associado a esta aula destina-se a tornar observável, no computador de cada aluno, a distinção entre virtualização completa e virtualização ao nível do sistema operativo. Utiliza Docker por simplicidade de instalação, sendo os mecanismos de núcleo subjacentes matéria da Aula 04.

- Duração estimada de sessenta a noventa minutos, sem necessidade de acesso ao servidor da unidade curricular.
- Entrega individual, constituída por seis observações registadas ao longo do guião.

## Laboratório: Percurso

---

- Distinção entre imagem e instância em execução, observável na primeira execução.
- Isolamento de processos, sistema de ficheiros e identificação de máquina.
- **Comparação do núcleo dentro e fora do contentor**, observação central do trabalho.
- Estrutura em camadas de uma imagem e reutilização entre imagens derivadas.
- Publicação de um serviço em rede e construção de imagem própria.
- Persistência de dados e imposição de limites de recursos.

## Pergunta 7

---

### Antecipação de um resultado do laboratório

Um aluno executa `uname -r` dentro de um contentor no seu computador com macOS e obtém a identificação de um núcleo Linux. Qual a interpretação correta?

- A) O contentor possui núcleo Linux próprio, independente do anfitrião.
- B) O Docker Desktop executa uma máquina virtual Linux sobre um hypervisor do sistema, e o contentor partilha o núcleo dessa máquina virtual.
- C) O resultado indica erro de instalação.
- D) O macOS possui um núcleo Linux interno utilizado pelos contentores.

## Resposta 7

---

### Opção correta: B

Os contentores constituem mecanismo do núcleo Linux e não são executáveis sobre o núcleo do macOS. O Docker Desktop resolve a questão instanciando uma máquina virtual Linux sobre o hypervisor do sistema, e é o núcleo dessa máquina virtual que o contentor partilha. Verifica-se assim, no mesmo equipamento, a coexistência dos dois modelos estudados: virtualização completa a sustentar a máquina virtual, e virtualização ao nível do sistema operativo no seu interior. Em Linux, a mesma verificação devolveria o núcleo do próprio anfitrião.

## Síntese da Aula (1/2)

---

- A virtualização aplica-se a três níveis distintos, diferenciados pela fronteira de duplicação entre instâncias: hardware, sistema operativo e aplicação.
- O teorema de Popek e Goldberg estabelece a condição de virtualizabilidade de uma arquitetura; o x86 não a satisfazia, do que decorreram três respostas distintas.
- Tradução binária, paravirtualização e assistência de hardware resolvem a mesma limitação, diferindo na necessidade de modificar o convidado e no suporte exigido ao processador.
- A virtualização de memória evoluiu das tabelas sombra, de manutenção dispendiosa, para a paginação em dois níveis assistida por hardware.

## Síntese da Aula (2/2)

---

- O I/O admite emulação, interface paravirtualizada virtio, atribuição exclusiva por passthrough ou partilha por SR-IOV, com compromissos entre desempenho e flexibilidade.
- O sobrecompromisso é admissível em memória, recurso reclamável, e exige vigilância acrescida em armazenamento, onde o espaço escrito não é recuperável.
- A migração por pré-cópia converge apenas quando a largura de banda excede a taxa de modificação de páginas.
- Máquinas virtuais e contentores diferem na fronteira de isolamento, do hardware ou do núcleo, o que determina a sua aplicação em cenários multi-cliente.

# Obrigada.

---

*Questões e comentários*